



BIT1123 OBJECT ORIENTED PROGRAMMING

ASSIGNMENT 1 - INDIVIDUAL (20%)

**FACULTY OF INFORMATION TECHNOLOGY CITY
UNIVERSITY MALAYSIA CYBERJAYA CAMPUS**

PREPARED BY SIR NAZMIRUL IZZAD BIN NASSIR

NAME:	FARUK OMAR
STUDENT ID:	202501010193
PASSPORT:	B00569058
PROGRAMME:	BCSSE

Self-Reflective Report

1. Introduction

Throughout this semester, I learned the fundamentals of Object-Oriented Programming (OOP) using Java through weekly tutorials. At the beginning, Java programming was challenging because I was still learning how classes, objects, and methods work together. By completing the tutorials step by step, I gradually became more confident in writing Java programs and understanding OOP concepts.

2. Knowledge Gained from Tutorials

Week 1 - Basic Java Programming

In Week 1, I practiced the basic structure of a Java program, the `main()` method and console output. The HelloWorld exercise helped me understand how a Java source file is structured and how the class name must match the file name. The StudentGrade exercise developed my understanding of keyboard input, variables, comparison operators and if-else decision making. It also showed me how a program can validate input and convert a numeric mark into a meaningful grade.

Week 2 - Classes, Objects, Constructors and Methods

Week 2 introduced the idea of representing a real-world entity as a class. In the Student class, attributes such as name, age and GPA were stored inside an object. I used a constructor to initialize the object and methods such as `displayInfo()`, `study()` and `takeExam()` to describe the behavior of the student. The Main class created a Student object using the `new` keyword and called its methods. This tutorial helped me understand the difference between a class, which acts like a blueprint, and an object, which is an actual instance created from that blueprint.

Weeks 3-4 - Inheritance, Method Overriding and Polymorphism

Weeks 3 and 4 expanded the program into a small class hierarchy. Person acted as the parent class, while Student and Lecturer inherited from Person. Both subclasses used `super()` to initialize inherited information and overrode the `introduce()` method. In Main, Person references were used for Person, Student and Lecturer objects, and each object

produced its own version of `introduce()`. This was my first clear example of runtime polymorphism. I learned that inheritance reduces repeated code, while method overriding allows subclasses to provide specialized behavior.

Week 5 - Encapsulation

Week 5 focused on encapsulation. The `Student` class stored `studentID`, `name`, `cgpa` and `programme` as private variables and exposed them through getter and setter methods. This tutorial helped me understand why fields should not always be public. By controlling access through methods, a class protects its internal state and creates a clearer boundary between the data and the code that uses it. The documentation task also made me explain why private variables and getters/setters are useful instead of only writing the code.

Week 6 - Inheritance with Employee and Lecturer

In Week 6, I practiced inheritance again using `Employee` as a base class and `Lecturer` as a derived class. `Employee` stored common information such as `id` and `name`, while `Lecturer` added `subject` and `department`. The `Lecturer` constructor called `super(id, name)` before assigning its own fields. This tutorial reinforced the purpose of inheritance and helped me understand access levels such as `protected`, where inherited classes can access shared data without making the fields completely public.

Week 7 - Abstraction

Week 7 introduced abstraction through the abstract `Appliance` class. The class contained common data and concrete methods such as `displayBrand()`, `turnOn()` and `turnOff()`, while `operate()` was declared abstract. In `Main`, different appliance objects supplied their own implementation of `operate()`. This showed me that an abstract class can define the common structure of a group of objects while leaving specific behavior to subclasses or implementations. I also understood that an abstract class cannot be instantiated directly unless its abstract behavior is implemented.

Weeks 8-9 - Collections and File Handling

Weeks 8 and 9 combined several practical Java features. I used an `ArrayList<String>` to store tasks entered by the user, `Scanner` for keyboard input, and loops to display the task list. I then used `BufferedWriter` and `FileWriter` to save the tasks into `task.txt` and `BufferedReader` and `FileReader` to load them again. The program handled `IOException` using try-catch blocks. This tutorial was useful because it connected program data with permanent storage and showed why exception handling is necessary when working with files.

Week 10 - Java Swing GUI and Event Handling

Week 10 introduced graphical user interface programming. I created a `Questions` class to store the quiz question, answer options and correct answer. The `QuizBattleGUI` class extended `JFrame` and implemented `ActionListener`. Labels and buttons were added to the window, and button clicks were processed through `actionPerformed()`. This tutorial changed the way I thought about program flow because the program waits for a user event instead of simply running line by line from start to finish. It also showed how OOP supports GUI development by separating quiz data from the interface logic.

I learned the basic structure of Java programs, including variables, input and output, classes, and methods. As the tutorials continued, I gained a better understanding of OOP concepts such as constructors, encapsulation, inheritance, and polymorphism. These concepts were difficult initially, but practicing different examples helped me understand how they are used to create organized and reusable programs.

3. Challenges Encountered

One challenge I faced was understanding the relationship between classes and objects. I was initially confused about why objects were needed and how different classes interact with each other. I also faced programming errors caused by syntax mistakes, incorrect logic, and missing code elements.

4. How the Challenges Were Overcome

I overcame these challenges by practicing more programs, reading error messages carefully, and reviewing tutorial examples. Instead of immediately changing code randomly, I learned to identify the cause of errors and solve problems step by step. I also learned how to organize my work using GitHub.

5. Improvement in Java Programming Skills

My Java programming skills improved throughout the semester. I became better at writing code, debugging errors, and applying OOP principles. I also learned that good programming is not only about making a program work, but also about writing code that is clear and maintainable.

6. Understanding of Object-Oriented Programming Concepts

This course helped me understand the importance of OOP in software development. Encapsulation helps protect data, inheritance allows code reuse, and polymorphism provides flexibility when designing programs. These concepts became clearer after applying them in practical exercises.

7. Future Learning Plans

In the future, I plan to improve my programming skills by developing larger Java projects and learning more advanced software development techniques. I also want to improve my ability to write cleaner code and maintain better project documentation.

8. Conclusion

Overall, this course gave me a strong foundation in Object-Oriented Programming using Java. Although I experienced difficulties, those challenges improved my problem-solving skills and confidence. Managing my GitHub repository also introduced me to professional programming practices.

9. GitHub Repository URL

https://github.com/alexfaruk86-star/Faruk-Omar_OOP-tutorials

Appendix A - Tutorial Portfolio Summary

Tutorial	Key Concept	Evidence in Repository
Week 1	Java basics and selection	HelloWorld.java, StudentGrade.java
Week 2	Classes and objects	Student.java, Main.java
Weeks 3-4	Inheritance and polymorphism	Person.java, Student.java, Lecturer.java, Main.java
Week 5	Encapsulation	Student.java, Main.java, documentation.txt
Week 6	Inheritance and super	Employee.java, Lecturer.java, Main.java
Week 7	Abstraction	Appliance.java, Main.java
Weeks 8-9	Collections and file I/O	Main.java, task.txt
Week 10	Swing GUI and events	Questions.java, QuizBattleGUI.java